

מדריך ייעודי: אלגוריתם המאצ'ינג, ניתוב AI וגשר MT5

UI_alfa - Matching Engine Guide

מדריך טכני מקיף: ניתוח מנגנון ההתאמה, חישובי עמלות, והסבר מפורט על
ניתוב INTERNAL (B-Book) מול EXTERNAL (A-Book)

מחלקה מרכזית: OrderExecutionService.java (com.brokerui.broker)

רכיבי ניתוב ואינטגרציה: NNPredictorClient.java, MT5IntegrationService.java, TradingFees.java

סיווג הניתוב: INTERNAL (B-Book) vs. EXTERNAL (A-Book MT5 Bridge)

תאריך עדכון: יולי 2026

1. פרק 1: מבוא וארכיטקטורת מנוע המאצ'ינג (Matching Engine Overview)
2. פרק 2: לופ המתזמן ומחזור החיים של הוראה (Scheduled Tick Loop)
3. פרק 3: אלגוריתם התאמת המחירים (Market, Limit, and Stop Matching Logic)
4. פרק 4: חישוב מרווחים (Spread) ועמלות (Commissions) בתהליך המאצ'ינג
5. פרק 5: מנוע ניתוב הבינה המלאכותית: INTERNAL (B-Book) vs. EXTERNAL (A-Book)
6. פרק 6: גשר MetaTrader 5 וביצוע חיצוני (A-Book MT5 Integration Bridge)
7. פרק 7: 5 תרחישי מאצ'ינג מעשיים (Matching Engine Case Studies & Simulator)

פרק 1: מבוא וארכיטקטורת מנוע המאצ'ינג (Matching Engine Overview)

מנוע ה-Matching Engine ב-UI_alfa אחראי לקבלת הוראות מסחר (BUY / SELL) מהמשתמשים, בדיקת תנאי השוק בלייב, התאמת המחירים (Order Matching), ביצוע פתיחת פוזיציות, גביית עמלות ומרווחים, וקבלת הכרעת ניתוב חכמה בעזרת אלגוריתם למידת מכונה (Neural Network Predictor) לניתוב פנימי (B-Book) או חיצוני (A-Book MT5 Bridge).

שירות הליבה: מנגנון המאצ'ינג מנוהל בתוך המחלקה OrderExecutionService.java, אשר רצה ברקע בלופ מתזמן מחזורי (@Scheduled(fixedDelay = 2000)).

1.1 רכיבי מנוע המאצ'ינג בפרויקט

שם המחלקה / הקובץ	נתיב במערכת	תפקיד במנוע המאצ'ינג
OrderExecutionService.java	com.brokerui.broker	מנוע ההתאמה הראשי: מריץ את לופ ה-Tick, בודק תנאי מילוי, ומבצע את העסקה.
TradingFees.java	com.brokerui.broker	מחשב את ה-Spread (מרווח קנייה/מכירה) והעמלות המנוכות בעת ביצוע המאצ'ינג.
NNPredictorClient.java	com.brokerui.broker	מתשאל שרת AI לקבלת הסתברות ניצחון והמלצת ניתוב (B-Book vs A-Book).
MT5IntegrationService.java	com.brokerui.broker	משדר את ההוראה שנמצאה מתאימה לניתוב חיצוני (A-Book) אל גשר MetaTrader 5.
demo_matching_cases.js	שורש הפרויקט	תסריט סימולציה מעשי המריץ 5 תרחישי מאצ'ינג לבדיקת המערכת.

פרק 2: לופ המתזמן ומחזור החיים של הוראה (Scheduled Tick Loop)

מנוע המאצ'ינג פועל בשיטת Scheduled Tick Loop הרץ מדי 2000 מילי-שניות (2 שניות). כלופ זה, השירות דג את 50 ההוראות הפעילות הוותיקות ביותר במצב NEW, ובודק מול ציטוטי המחיר העדכניים אם התקיימו התנאים לביצוע המאצ'ינג.

2.1 קוד מפתח: לופ התזמן של מנוע המאצ'ינג (OrderExecutionService.java)

```
/**
 * Periodic scheduler tick running every 2 seconds.
 * Scans for pending NEW orders and attempts execution.
 */
@Scheduled(fixedDelay = 2000)
public void tick() {
    List<BrokerOrder> open = orderRepo.findTop50ByStatusOrderByCreatedAtAsc("NEW");
    for (BrokerOrder o : open) {
        try {
            self.tryExecute(o.getId());
        } catch (Exception ignored) {}
    }
    try {
        checkPositionsSlTp();
    } catch (Exception ignored) {}
}
```

2.2 זרימת תהליך המאצ'ינג (Step-by-Step Order Lifecycle)

1. **קבלת ההוראה:** הלקוח מזין הוראה ב-Frontend (בקר BrokerApiController.java), וההוראה נשמרת בסטטוס NEW.
2. **דלייט מחיר בלייב:** כלופ ה-Tick, השירות מקבל את מחיר השוק העדכני ביותר מ-MarketPriceService.java.
3. **בדיקת תנאי המאצ'ינג:** הפונקציות shouldFillLimit ו-shouldFillStop מכריעות אם המחיר הגיע למחיר היעד.
4. **החלת Spread ועמלות:** מחשבים את מחיר הביצוע הסופי בעזרת TradingFees.applySpread, ומנכים עמלה ב-TradingFees.calculateCommission.
5. **פתיחת פוזיציה והקצאת ביטחונות:** מעדכנים את TradingAccount ויוצרים ישות Position חדשה.
6. **ניתוב AI ו-MT5:** מתשאולים את NNPredictorClient לקביעת ניתוב B-Book או A-Book.

פרק 3: אלגוריתם התאמת המחירים (Market, Limit, and Stop Matching Logic)

מנוע המאצ'ינג תומך בשלושה סוגי הוראות מסחר עיקריים:

- **Market Order (הוראת שוק):** מבוצעת באופן מיידי מול מחיר השוק הנוכחי (shouldFill = true).
- **Limit Order (הוראת הגבלה):** מבוצעת רק כאשר מחיר השוק משתפר מעבר למחיר היעד (BUY במחיר נמוך או שווה ל-Limit, SELL במחיר גבוה או שווה ל-Limit).
- **Stop Order (הוראת קטעי הפסד / פריצה):** מבוצעת כאשר מחיר השוק חוצה את מחיר ה-Stop (BUY במחיר גבוה או שווה ל-Stop, SELL במחיר נמוך או שווה ל-Stop).

3.1 קוד מפתח: מנגנון בדיקת תנאי הביצוע (OrderExecutionService.java)

```
@Transactional
public void tryExecute(Long orderId) {
    BrokerOrder order = orderRepo.findById(orderId).orElse(null);
    if (order == null || !"NEW".equalsIgnoreCase(order.getStatus())) return;

    String symbolCode = order.getSymbolCode();
    Symbol sym = symbolRepo.findByCode(symbolCode).orElse(null);
    if (sym == null || !sym.isEnabled()) {
        order.setStatus("REJECTED");
        orderRepo.save(order);
        return;
    }

    BigDecimal last;
    try {
        double price = priceService.getLivePrice(symbolCode);
        if (price <= 0) return; // keep NEW
        last = BigDecimal.valueOf(price);
    } catch (Exception e) {
        return; // keep NEW
    }

    String side = order.getSide() == null ? "" : order.getSide().trim().toUpperCase();
    String type = order.getOrderType() == null ? "" : order.getOrderType().trim().toUpperCase();

    // אלגוריתם ההכרעה לתאמת מחיר
    boolean shouldFill = (type.equals("MARKET")) ||
        switch (type) {
            case "LIMIT" -> shouldFillLimit(order, side, last);
            case "STOP" -> shouldFillStop(order, side, last);
            default -> false;
        };

    if (!shouldFill) return;

    executeFilled(order, last);
}

private static boolean shouldFillLimit(BrokerOrder o, String side, BigDecimal last) {
    BigDecimal limit = o.getLimitPrice();
    if (limit == null) return false;
    if ("BUY".equals(side)) return last.compareTo(limit) <= 0;
    if ("SELL".equals(side)) return last.compareTo(limit) >= 0;
    return false;
}

private static boolean shouldFillStop(BrokerOrder o, String side, BigDecimal last) {
    BigDecimal stop = o.getStopPrice();
    if (stop == null) return false;
    if ("BUY".equals(side)) return last.compareTo(stop) >= 0;
    if ("SELL".equals(side)) return last.compareTo(stop) <= 0;
}
```

```
    return false;  
}
```

פרק 4: חישוב מרווחים (Spread) ועמלות (Commissions)

בתהליך המאצ'ינג

במהלך ביצוע המאצ'ינג, מחיר השוק הגולמי (Raw Price) אינו המחיר הסופי שבו מתבצעת העסקה. המחלקה TradingFees.java מפעילה מנגנון דינמי להחלת מרווח Ask/Bid Spread ולחישוב עמלת מסחר המנוכה מחשבון הלקוח.

$$\text{Ask Price (BUY)} = \text{Raw Price} \times (1 + \text{SpreadRate} / 2)$$

$$\text{Bid Price (SELL)} = \text{Raw Price} \times (1 - \text{SpreadRate} / 2)$$

4.1 קוד מפתח: חישוב ה-Spread והעמלות (TradingFees.java)

```
package com.brokerui.broker;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class TradingFees {
    private static final BigDecimal DEFAULT_SPREAD_RATE = new BigDecimal("0.0002"); // 0.02%
    private static final BigDecimal COMMISSION_RATE = new BigDecimal("0.0005"); // 0.05%

    public static BigDecimal applySpread(BigDecimal rawPrice, boolean isBuy) {
        if (rawPrice == null || rawPrice.compareTo(BigDecimal.ZERO) <= 0) return rawPrice;
        BigDecimal halfSpread = rawPrice.multiply(DEFAULT_SPREAD_RATE).divide(new BigDecimal("2"), 6,
        return isBuy ? rawPrice.add(halfSpread) : rawPrice.subtract(halfSpread);
    }

    public static BigDecimal calculateCommission(String symbol, BigDecimal qty, BigDecimal price) {
        if (qty == null || price == null) return BigDecimal.ZERO;
        BigDecimal notional = qty.multiply(price);
        return notional.multiply(COMMISSION_RATE).setScale(2, RoundingMode.HALF_UP);
    }

    public static void charge(TradingAccount ta, BrokerOrder order, BigDecimal commission) {
        if (commission.compareTo(BigDecimal.ZERO) > 0) {
            ta.setBalance(ta.getBalance().subtract(commission));
            order.setCommission(commission);
        }
    }
}
```


פרק 5: מנוע ניתוב הבינה המלאכותית: INTERNAL (B-Book) vs. EXTERNAL (A-Book)

לאחר שמוראת מסחר הגיעה לתנאי ביצוע (Filled), מנוע המאצ'ינג משתמש במודל הבינה המלאכותית ב- NNPredictorClient.java כדי לקבוע את מודל הסיכון והניתוב הכלכלי של העסקה.

מה המשמעות של INTERNAL מול EXTERNAL בפריקט?

הבחירה בין INTERNAL ל- EXTERNAL קובעת מי מנהל את הסיכון של העסקה והיכן נרשמת הנזילות!

5.1 טבלת השוואה מקיפה: INTERNAL vs. EXTERNAL

מאפיין	ניתוב פנימי - INTERNAL (B-Book)	ניתוב חיצוני - EXTERNAL (A-Book)
הסיווג הכלכלי	B-Book Execution : הברוקר משמש כצד הנגדי לעסקה בתוך הפלטפורמה.	A-Book Execution : העסקה משודרת החוצה לכיסוי וגידור בשוק הבינלאומי.
שיקול מודל ה-AI	תנועת מסחר של סוחר קמעונאי / ניטרלי ($routeRecommendation > 0.5$). הסתברות הפסד גבוהה לסוחר.	עסקת סיכון גבוה / סוחר רווחי ($routeRecommendation \leq 0.5$). הברוקר מעדיף לגדר את הסיכון.
שידור לגשר MT5	לא משודר ל-MT5 Bridge . העסקה נשמרת בתוך מסד הנתונים של השרת בלבד.	משודר מיידית ל-MT5 Bridge ב- MT5IntegrationService.java לשוק החופשי.
רווח הברוקר	הברוקר מרוויח את ה-Spread + העמלה + הפסד הלקוח במידה והפוזיציה נסגרת בהפסד.	הברוקר מרוויח עמלת ברוקראז' נקי בלבד, תוך העברת הסיכון לספק הנזילות החיצוני.
הודעת הלוג בשרת	[AI ROUTING - INTERNAL (B-BOOK)] Order #123 matched internally. Skipped MT5 routing.	[AI ROUTING - EXTERNAL (A-BOOK)] Order #124 routed to MetaTrader 5 Bridge.

5.2 קוד מפתח: אלגוריתם הניתוב ב-OrderExecutionService.java

```
// 1. AI-למודל ה (Features Vector) הרכבת 8 המאפיינים
double[] features = {
    buyQtyNorm, sellQtyNorm, spreadNorm, imbalance,
    midPriceNorm, bookDepthBuy, bookDepthSell, 0.72
};

// 2. AI ROUTING DECISION LOGIC
java.util.Map<String, Object> pred = nnPredictorClient.getPrediction(features);
if (pred != null) {
    double matchProb = ((Number) pred.get("matchProb")).doubleValue();
    double expectedSavings = ((Number) pred.get("expectedSavings")).doubleValue();
    double routeRecommendation = ((Number) pred.get("routeRecommendation")).doubleValue();

    order.setNnMatchProb(matchProb);
    order.setNnExpectedSavings(BigDecimal.valueOf(expectedSavings));

    if (routeRecommendation > 0.5) {
        // INTERNAL (B-Book): Retain trade internally in platform liquidity pool
        order.setNnRouteRecommendation("INTERNAL");
        routeExternal = false;
    } else {
        // EXTERNAL (A-Book): Hedge trade externally via MetaTrader 5 Bridge
        order.setNnRouteRecommendation("EXTERNAL");
        routeExternal = true;
    }
}

// 3. AI-ביצוע הניתוב לפי הכרעת ה
if (routeExternal) {
    // EXTERNAL ROUTING: Forward trade to MetaTrader 5 Bridge
    System.out.println(String.format(java.util.Locale.US,
        "[AI ROUTING - EXTERNAL (A-BOOK)] Order #%d for %s %s %.4f @ %.4f routed to MetaTrader 5 Bridge",
        order.getId(), order.getSide(), order.getSymbolCode(), qty.doubleValue(), price.doubleValue()));
    mt5Service.sendTrade(order.getSymbolCode(), order.getSide(), price.doubleValue(), tp, sl, qty.doubleValue());
} else {
    // INTERNAL ROUTING: Retain trade internally on platform ledger
    System.out.println(String.format(java.util.Locale.US,
        "[AI ROUTING - INTERNAL (B-BOOK)] Order #%d for %s %s %.4f @ %.4f matched internally. Skipped",
        order.getId(), order.getSide(), order.getSymbolCode(), qty.doubleValue(), price.doubleValue()));
}
```

פרק 6: גשר MetaTrader 5 וביצוע חיצוני (A-Book MT5) (Integration Bridge)

כאשר אלגוריתם ה-AI מכריע כי ההוראה מיועדת לניתוב חיצוני (EXTERNAL / A-Book), מנוע המאצ'ינג מעביר את נתוני העסקה אל גשר ה-MT5 ב-MT5IntegrationService.java וב-MT5ConnectionManager.java.

6.1 קוד מפתח: שידור עסקה לגשר (MT5IntegrationService.java)

```
package com.brokerui.broker;

import org.springframework.stereotype.Service;

@Service
public class MT5IntegrationService {
    private final MT5ConnectionManager connectionManager;

    public MT5IntegrationService(MT5ConnectionManager connectionManager) {
        this.connectionManager = connectionManager;
    }

    public void sendTrade(String symbol, String side, double price, double tp, double sl, double qty) {
        String payload = String.format(java.util.Locale.US,
            "TRADE|SYMBOL=%s|SIDE=%s|PRICE=%.4f|TP=%.4f|SL=%.4f|QTY=%.4f",
            symbol, side, price, tp, sl, qty);

        System.out.println("Forwarding trade to MT5 Bridge: " + payload);
        connectionManager.send(payload);
    }
}
```

6.2 מנהל החיבורים לגשר (MT5ConnectionManager.java)

```
@Service
public class MT5ConnectionManager {
    private final MT5JavaTradeWriter tradeWriter;

    public MT5ConnectionManager(MT5JavaTradeWriter tradeWriter) {
        this.tradeWriter = tradeWriter;
    }

    public void send(String payload) {
        try {
            tradeWriter.writeTradePayload(payload);
        } catch (Exception e) {
            System.err.println("Error writing payload to MT5 pipe: " + e.getMessage());
        }
    }
}
```

פרק 7: 5 תרחישי מאצ'ינג מעשיים (Matching Engine Case)

(Studies & Simulator)

לצורך בדיקת מנוע המאצ'ינג, נכתב תסריט סימולציה מקיף ב-demo_matching_cases.js המדגים 5 מקרי בוחן מעשיים:

7.1 פירוט 5 מקרי הבוחן בסימולטור

מספר תרחיש	שם התרחיש (Case Study)	תיאור והתנהגות מנוע המאצ'ינג
Case 1	Exact Price Match	הקונה מציע \$100 והמוכר דורש \$100. מנוע המאצ'ינג מבצע התאמה מלאה מיידית.
Case 2	Price Improvement Match	הקונה מוכן לשלם עד \$105 והמוכר מציע ב-\$100. המאצ'ינג מתבצע במחיר השוק הטוב יותר (\$100).
Case 3	Partial Execution Match	קנייה של 10 יחידות מול מכירה של 4 יחידות. המאצ'ינג מבצע 4 יחידות, והיתרה (6) נשארת ב-NEW.
Case 4	Spread Disparity (No Match)	קנייה ב-\$95 מול מכירה ב-\$105. אין התאמת מחירים - ההוראות נשארות ממתונות ב-NEW.
Case 5	SL/TP Auto-Trigger Match	מחיר השוק מגיע ל-Take Profit או Stop Loss של פוזיציה קיימת, ומנוע המאצ'ינג סוגר אותה אוטומטית.

7.2 תסריט הסימולציה ב-JavaScript (demo_matching_cases.js)

```
const cases = [
  {
    id: 1,
    title: "Exact Price Match",
    buyPrice: 100.0,
    sellPrice: 100.0,
    expectedStatus: "FILLED"
  },
  {
    id: 2,
    title: "Price Improvement Match",
    buyPrice: 105.0,
    sellPrice: 100.0,
    expectedStatus: "FILLED at 100.0"
  },
  {
    id: 3,
    title: "Partial Execution Match",
    buyQty: 10.0,
    sellQty: 4.0,
    expectedStatus: "PARTIALLY_FILLED (4/10)"
  },
  {
    id: 4,
    title: "Spread Disparity (No Match)",
    buyPrice: 95.0,
    sellPrice: 105.0,
    expectedStatus: "PENDING (NEW)"
  },
  {
    id: 5,
    title: "SL/TP Auto-Trigger Match",
    triggerPrice: 110.0,
    takeProfit: 110.0,
    expectedStatus: "CLOSED by TP"
  }
];
```

סוף מדריך אלגוריתם המאצ'ינג - UI_alfa Matching Engine Guide